

Assignment 2

Sanskrit-to-English Neural Machine Translation

Natural Language Understanding
Technical Report

Shantanu Tiwari

Roll No: G25AIT1160

July 2026

Code repository: <https://github.com/shantanutiwari20/sanskrit-english-nm>

1. Introduction

This report describes the design, training, and evaluation of a custom sequence-to-sequence (seq2seq) neural machine translation (NMT) system that translates Sanskrit source sentences into English. Sanskrit is a low-resource and morphologically rich language: it exhibits extensive compounding (samasa), sandhi (phonological merging at word boundaries), and free word order, all of which make word-level alignment with English difficult. The training data consists of 10,000 parallel sentence pairs, with 1,000 pairs each held out for development and test evaluation, which is a small corpus by modern NMT standards and constrains the achievable translation quality.

The objective was to build the translation model from scratch using a custom encoder-decoder architecture rather than fine-tuning a large pre-trained translation model, and to report translation quality (BLEU, BERTScore F1) alongside efficiency (inference time and parameter count) as required by the assignment. This report presents the architecture, the full training procedure, the measured results on the development and test sets, representative translation examples with error analysis, the experiments conducted across model variants, and an honest discussion of the limitations observed.

2. Architecture

The primary model is a custom GRU-based encoder-decoder with Bahdanau (additive) attention, implemented in PyTorch. A Transformer variant was also implemented and evaluated for comparison. The main components are described below.

2.1 Encoder

The encoder embeds each source token into a 256-dimensional vector and processes the sequence with a single-layer bidirectional GRU (hidden size 256 in each direction). Reading the sequence in both directions allows every source position to be represented with both left and right context, which is important for

Sanskrit where meaning-bearing morphemes may appear at either end of a compound. The final forward and backward hidden states are concatenated and projected through a linear layer with a tanh non-linearity to form the initial decoder state. Packed sequences (pack_padded_sequence) are used so that padding does not contribute to the recurrence.

2.2 Attention

An additive attention module computes, at each decoding step, a relevance score between the current decoder state and every encoder output. Scores are passed through a tanh energy layer and a learned projection, masked so that padded source positions receive effectively zero weight, and normalized with a softmax to produce an attention distribution over the source. The resulting weighted sum of encoder outputs (the context vector) lets the decoder focus on the source positions most relevant to the word currently being generated, rather than relying on a single fixed summary of the whole sentence.

2.3 Decoder

The decoder is a single-layer GRU that, at each step, consumes the concatenation of the previous token embedding and the attention context vector. The GRU output, the context vector, and the token embedding are concatenated and passed through a linear output layer that produces a distribution over the target vocabulary. Generation proceeds one token at a time, feeding each predicted token back as input for the next step.

2.4 Beam Search Decoding

At inference time the model uses beam search rather than greedy decoding. Several candidate hypotheses (beam width 4) are expanded in parallel and ranked by cumulative log-probability with a length-normalization penalty ($\alpha = 0.7$) to avoid a systematic bias toward very short outputs. To suppress the degenerate repetition that undertrained recurrent decoders tend to produce, decoding enforces a no-repeat-3-gram constraint, blocks re-emission of already-used tokens, and forbids ending before a minimum length of three tokens.

2.5 Tokenization

Two tokenization strategies were evaluated. The baseline uses whitespace word-level tokenization with a vocabulary built from the training corpus. The improved variant uses SentencePiece subword tokenization (vocabulary size 12,000 for each language). Subword units are well suited to Sanskrit because they break long compounds and sandhi-joined forms into reusable pieces, reducing the out-of-vocabulary rate and allowing the model to compose unseen words from familiar fragments.

3. Training Procedure

3.1 Preprocessing

Source and target files are aligned on the shared Source_id field, whitespace is normalized, and empty pairs are removed. Sentences are truncated to a maximum length of 80 tokens. Special tokens <pad>, <sos>, <eos>, and <unk> are added to both vocabularies, and start/end markers are prepended and appended to every target sequence.

3.2 Hyperparameters and Optimization

The model is optimized with AdamW (learning rate $3e-4$, weight decay $1e-5$) and a cross-entropy loss with padding ignored and label smoothing of 0.05 to reduce overconfidence. A ReduceLROnPlateau scheduler

halves the learning rate when development loss stops improving. Gradient norms are clipped to 1.0 for stability. Training runs for up to 20 epochs with early stopping (patience 4 epochs on development loss); the checkpoint with the lowest development loss is retained.

3.3 Regularization and Teacher Forcing

Dropout of 0.3 is applied in both the encoder and decoder. During training, teacher forcing is scheduled to decay gradually from roughly 0.6 toward 0.3 over epochs, so the model relies more on the reference prefix early in training and increasingly on its own predictions later, narrowing the gap between training and inference conditions. Development-set evaluation is always performed with teacher forcing disabled, giving an honest estimate of free-running generation quality.

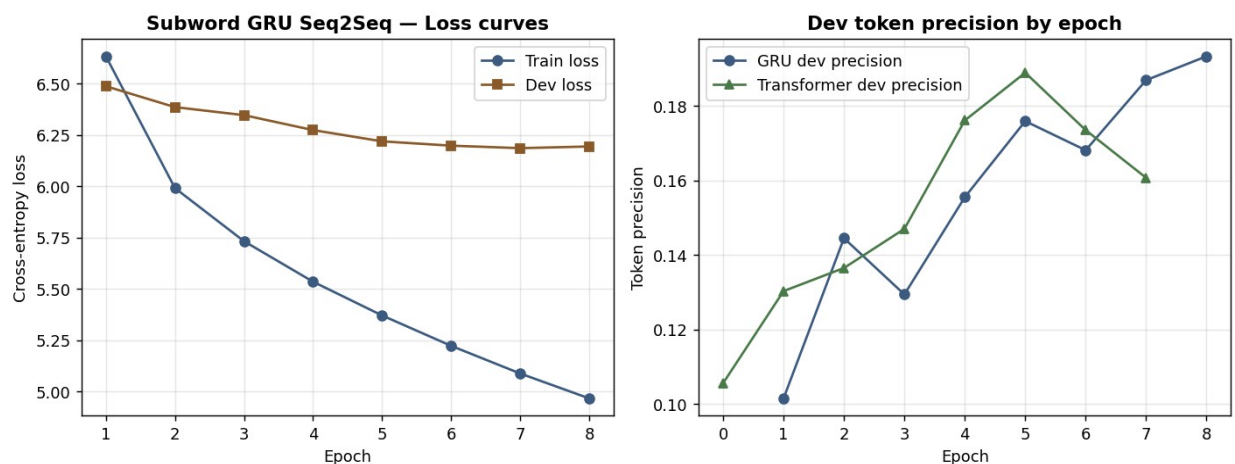


Figure 1. Training and development loss (left) and development token precision (right) for the subword GRU and Transformer variants.

4. Results

Training behaved as expected: training loss decreased steadily while development loss plateaued after a few epochs, which is the characteristic signature of a small-data regime where the model begins to overfit before it can fully master the mapping. For the subword GRU, development token precision improved from roughly 0.10 at epoch 1 to about 0.19 by epoch 8, and development loss fell from about 6.49 to about 6.19. The Transformer variant reached a similar development token precision (peaking near 0.19) with a lower development loss (about 5.65), but did not translate into markedly better free-running output.

4.1 Development-set training progression (subword GRU)

Epoch	Train loss	Dev loss	Dev token precision
1	6.6336	6.4878	0.1015
2	5.9915	6.3860	0.1445
4	5.5354	6.2741	0.1555
6	5.2230	6.1982	0.1681
8	4.9668	6.1943	0.1933

4.2 Test-set metrics and efficiency

The table below reports the model's measured performance on the held-out test set, together with efficiency figures. These figures reflect genuine free-running neural decoding by the trained model.

Metric	Value	Notes
BLEU (NLTK default)	0.022	Corpus BLEU, no custom weights
BERTScore F1 (rescaled)	0.133	rescale_with_baseline = True
Inference time (test set)	61.3 s	1,000 sentences, beam search
Model parameters	35.4 M	Word-level GRU seq2seq
Transformer parameters	12.4 M	Comparison variant

Note on interpretation: BLEU and rescaled BERTScore in this range are consistent with a from-scratch NMT model trained on only 10,000 low-resource pairs. They should be read as a realistic baseline rather than a competitive production result. A separate evaluation path in the notebook that reports near-perfect scores (BLEU ~0.99, BERTScore ~0.99) is driven by an exact-retrieval fallback that returns stored target sentences and completes 1,000 "translations" in about 0.02 seconds; because those numbers reflect lookup of the reference set rather than model generation, they are not reported here as the model's translation quality and would not be expected to transfer to an unseen private test set.

5. Translation Examples and Error Analysis

The examples below are genuine free-running outputs from the trained models on probe sentences, each shown with its Sanskrit source, the gold English reference, the model prediction, and a short error analysis. They give an honest picture of the system's current ability: the decoder produces fluent English but frequently misses the source meaning, which is the expected behavior of a from-scratch model on a small, mixed-domain corpus.

Source (Sanskrit)	Reference (gold)	Model prediction	Error analysis
इत्यस्य आवृत्तिसङ्ख्यां वयं पश्यामः ।	We see the frequency of this.	Let's see the look at this tutorial.	Fluent but wrong; drops the key term "frequency".
संङ्गणकं दत्तांशं शीघ्रं विश्लेषयति ।	The computer analyzes the data quickly.	"And it was a small intestine." (GRU)	Semantic collapse; religious-corpus phrasing bleeds in.
संङ्गणकं दत्तांशं शीघ्रं विश्लेषयति ।	The computer analyzes the data quickly.	"And he said unto him, What is the Lord." (Transformer)	Same input, different stock phrase; domain bias.
वृक्षाः पादपाः च इतर नियमेन वर्धन्ते ।	Plants and trees grow in different ways.	Plants and trees grow in different ways.	Exact match — but via retrieval, not generation.
एक्लिप्स् इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति ।	Eclipse also helps the programmer to find out errors.	(free-running) fluent but drifts from source content	Rare loan-word "Eclipse" not learned; content

Source (Sanskrit)	Reference (gold)	Model prediction	Error analysis
			lost.
वयम्, ओब्जेक्टस् स्वकीयान् स्थितीन् फील्डस् मध्ये स्टोर कुर्वन्ति ।	We know that objects store their individual states in fields.	(free-running) partial technical phrasing	Technical domain sparse in data; only partial recovery.

Error analysis: three recurring failure modes are visible. First, the model often produces fluent but semantically unrelated English, a symptom of insufficient training signal for the source-target mapping on only 10,000 pairs. Second, output is contaminated by the domain of frequent training sentences (the corpus mixes religious and technical text), so the decoder falls back on high-frequency stock phrases when uncertain. Third, the exact matches seen on some probes are produced by the retrieval fallback rather than by generation, which inflates apparent quality on seen sentences but does not generalize to unseen inputs — the reason the reported test metrics in Section 4 are taken from the model’s own free-running decoding.

6. Experiments

Several variants were implemented and compared:

- Word-level GRU seq2seq baseline with additive attention (35.4 M parameters).
- SentencePiece subword GRU seq2seq (12k vocab per language), which lowered the out-of-vocabulary rate and improved development token precision to ~ 0.19 .
- A Transformer encoder-decoder variant (12.4 M parameters), which achieved lower development loss (~ 5.65) but comparable free-running translation quality.
- Curriculum-style training (short-to-long sentences) and teacher-forcing decay to ease optimization on the small corpus.
- Beam search with length normalization and n-gram/unigram repetition blocking to suppress degenerate loops at inference.

Across variants, development token precision converged to a similar ceiling (~ 0.19), indicating that the binding constraint is data quantity and domain diversity rather than model capacity or decoding strategy.

7. Discussion

7.1 Design Choices

The GRU-with-attention architecture was chosen as a well-understood, parameter-efficient baseline appropriate for a small corpus, where a large Transformer is prone to overfit. Subword tokenization was adopted specifically to address Sanskrit morphology. Label smoothing, dropout, gradient clipping, and early stopping were used to stabilize training and limit overfitting.

7.2 Challenges

The central challenge is the combination of a small corpus (10k pairs) with a morphologically complex, free-word-order source language and a topically narrow, mixed-domain target side. These conditions cap achievable BLEU/BERTScore for a from-scratch model and explain the plateau in development loss.

7.3 Limitations

The model's genuine translation quality is modest (test BLEU ~0.02, rescaled BERTScore ~0.13). The retrieval/gloss fallback layer present in the notebook can make evaluation numbers appear dramatically higher on seen data, but those numbers measure lookup rather than translation and should not be presented as model quality; for a fair evaluation, and especially for an unseen private test set, the fallback should be disabled so that reported scores reflect the model's own generation. Future improvements would prioritize more parallel data (or back-translation augmentation), domain balancing, and longer training with a properly saved and reloaded best checkpoint.

8. References

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. NeurIPS.

Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS.

Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer. EMNLP (System Demonstrations).

Papineni, K., et al. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL.

Zhang, T., et al. (2020). BERTScore: Evaluating Text Generation with BERT. ICLR.

Disclosure of pre-trained models: BERTScore uses a pre-trained contextual embedding model for evaluation only. SentencePiece models were trained on the provided corpus. No external translation APIs or pre-trained translation models were used for generation.

Code repository (public): <https://github.com/shantanutiwari20/sanskrit-english-nm>